

COL724/7524 Assignment 2, Part A: Link Scheduling

Mohit and teammate

1. The four policies as implemented

Queue and threads (A5). An acceptor thread only calls `accept()`. Each connection goes to a short-lived admission thread that reads the header line (5 s receive timeout), answers HEALTH immediately with the current queue depth, validates the file name (A25), and enqueues the request. For a GET it opens the file and takes the size from `fstat`, so the size is known, and fixed to one version of the file, when the request enters the queue. *server_threads* workers only loop on `next()`, then `serve_slice()` for one round, then either requeue the request (PREEMPTED) or finish it. Because admission never waits for a worker, requests really do pile up in the scheduler queue: a test that pins the only worker on a stalled PUT sees HEALTH report a depth of 6, and HEALTH still answers in about 1 ms while a silent client is connected.

Policies. *fcfs* serves in arrival order. *sjf* serves the queued request with the smallest declared byte count (file size for GET, the count in the request line for PUT: one key for both verbs), earliest arrival first on ties, with no aging. *fcfs* and *sjf* never preempt, so every request takes one round (*rounds* = 1). *rr* gives each scheduled request an allowance of $Q = 8192$ bytes per round and requeues it at the tail. *drr* uses the same queue but each request carries a deficit counter (initially 0): a round's allowance is deficit + Q , and whatever part of it goes unused is kept in the deficit instead of being forfeited; the deficit is discarded when the request finishes.

One round (serve_slice). A GET is sent in whole lines (A6): the line length is found by scanning to the next '\n' (a last line without one still counts), and lines are grouped `--p` per `write()` (A8) without changing which bytes go out or how the allowance is charged. A round ends as soon as the next line would exceed the remaining allowance (A13); under *rr* the unused remainder is added to *forfeited_bytes*, under *drr* it becomes the deficit. **A14** (*rr* only): if a line longer than Q starts a round, i.e. the round has transferred nothing yet, it is sent in full and the round ends, overrunning the allowance and logging one line. I read A14 this way because its stated purpose is to stop a request being requeued having transferred nothing, forever; a long line reached *mid-round* is handled by A13 instead (round ends, remainder forfeited) and goes out first in the next round. *drr* has no escape: the deficit grows by Q per round until it covers the line (A16), so a 20,000-byte line costs up to 3 rounds. A PUT round reads exactly `min(allowance, remaining)` bytes (A9, A13) into a temporary file that is renamed onto the destination when the last byte arrives, so a concurrent GET never sees a half-written file. The 'OK n' response line is protocol overhead and is not charged to the allowance; the terminating '\n' of every line is (A7).

Preemption state (A17). Everything a later round needs lives in the Request: the byte offset; the file descriptor opened at admission (so every round reads the version that was sized); the bytes already read from the socket but not yet consumed (for a PUT, the body bytes that arrived with the header); the deficit; *rounds* and *forfeited_bytes*; and the temporary-file path. Correctness is checked by 72 unit assertions that drive `serve_slice` over a message-preserving socketpair against rounds, forfeiture and A14 counts worked out by hand (e.g. ten 10-byte lines with $Q = 35$: *rr* takes 4 rounds and forfeits 15 bytes, *drr* takes 3 and forfeits 0), including a GET that keeps serving the old file after a PUT has replaced it, and by an integration suite that runs all four policies end to end.

2. Experimental setup

Machine and configuration. Server and client run on one laptop under WSL2 (Ubuntu 24.04, 8 logical CPUs) over loopback TCP, so absolute times are not network-representative; only comparisons between policies are meaningful. The reference configuration is 4 server threads and 8 client threads; the second is 1 server thread. $Q = 8192$ bytes. Each run makes 1000 counted requests from a closed loop of 8 client threads, after 3 uncounted seeding PUTs (dropped by sorting the CSV by arrival and removing the first 3 rows). Each request picks one of the workload files uniformly and does a GET or a PUT with probability 1/2. Percentiles are nearest-rank. `TCP_NODELAY` is set so every `write()` is a separate segment.

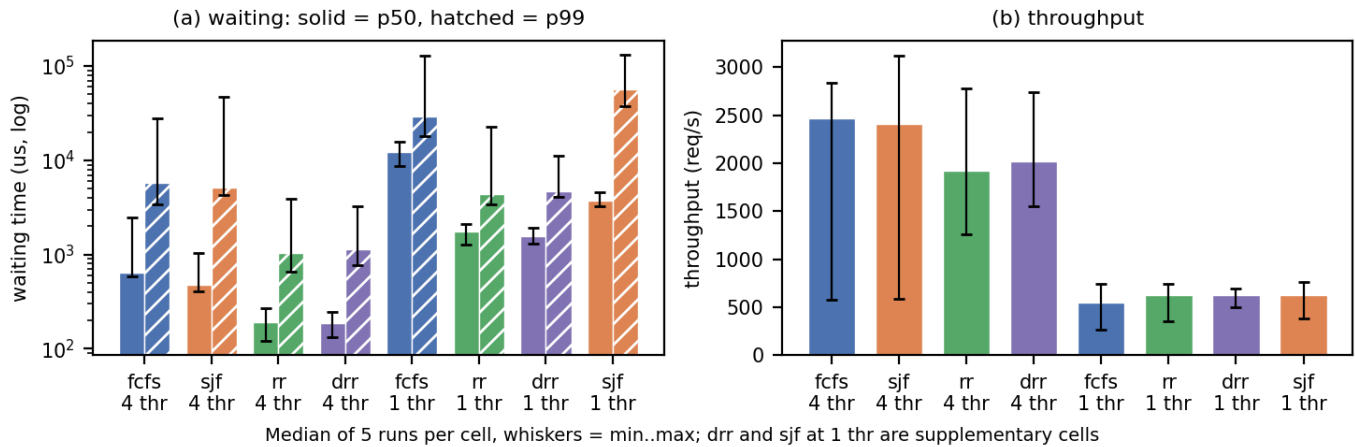
Workload (A27), both axes in one directory. *Size*: small.txt 1,044 B, medium.txt 30,774 B, large.txt 153,613 B. *Line length*: small and medium contain only ordinary 60-79 byte lines; large.txt has the same ordinary lines plus 6 lines of 20,000 bytes ($2.4 \times Q$) spread evenly through it, so it is both the large file and the long-line file. With $Q = 8192$ the large file needs 19 rounds under a byte-exact accounting.

Repetition. A28 asks for one run per cell. This setup is noisy from run to run (background activity on the Windows host made the slowest run of a cell up to 5 times slower than its fastest), so every cell was run **5 times**, in interleaved blocks (each block runs all cells once, preceded by an unrecorded warm-up run because the first cell of a session otherwise pays a cold-start cost). Tables report the **median over the 5 runs**, with [min-max] where useful; whiskers in the figures are min-max. Raw per-request CSVs of every run are in `results/run1..run5`. Two extra cells, *sjf* and *drr* at 1 server thread, are supplementary (not among the six required cells) and are marked as such.

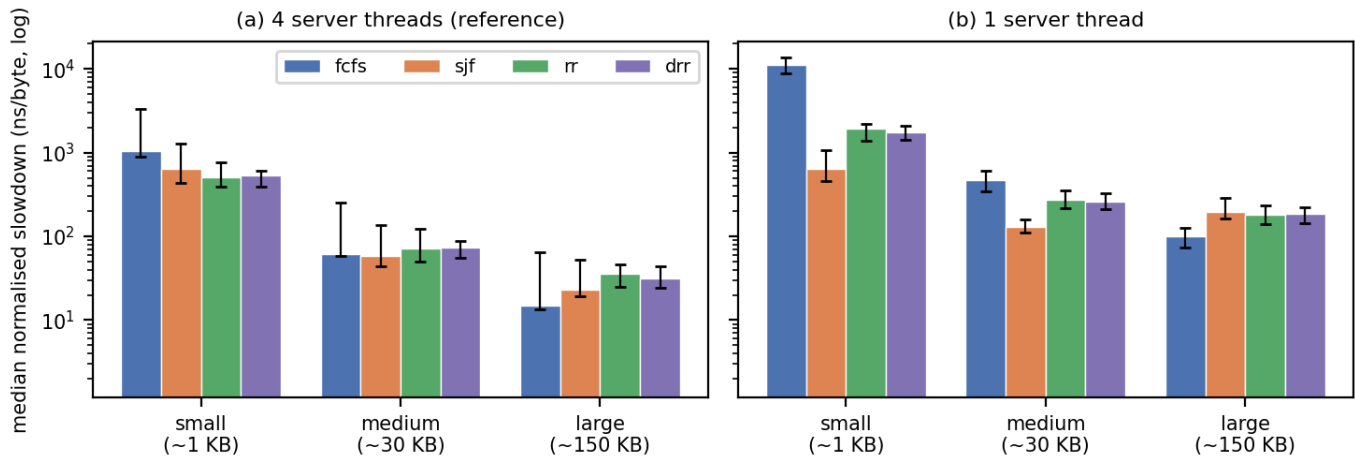
3. Per-run results

Table 1. Waiting time (start - arrival) and throughput, median [min-max] over 5 runs.

cell	server thr.	waiting p50 (us)	waiting p99 (us)	throughput (req/s)
fcfs	4	649 [591-2490]	5801 [3381-28152]	2471 [582-2835]
sjf	4	481 [404-1035]	5135 [4261-47210]	2412 [589-3124]
rr	4	190 [122-272]	1037 [653-3888]	1923 [1261-2780]
drr	4	187 [132-245]	1145 [766-3239]	2022 [1550-2738]
fcfs	1	12294 [8630-15858]	29384 [18200-127727]	545 [264-743]
rr	1	1761 [1289-2092]	4373 [3416-22504]	631 [358-746]
drr (suppl.)	1	1572 [1296-1911]	4643 [4109-11213]	630 [503-691]
sjf (suppl.)	1	3734 [3270-4562]	56825 [37316-131896]	626 [379-762]

**Table 2.** Normalised slowdown (response / bytes, ns per byte) by size class: median over runs of each run's median and p99. The first four rows are the reference configuration.

cell	small med	small p99	medium med	medium p99	large med	large p99
fcfs, 4 thr	1037	8410	62	326	15	84
sjf, 4 thr	644	3454	58	259	23	73
rr, 4 thr	502	1811	72	306	36	90
drr, 4 thr	531	3358	73	339	31	105
fcfs, 1 thr	11067	28366	468	960	101	228
rr, 1 thr	1924	6325	275	728	181	441
drr, 1 thr (suppl.)	1730	6568	260	734	187	460
sjf, 1 thr (suppl.)	641	5476	131	392	196	419



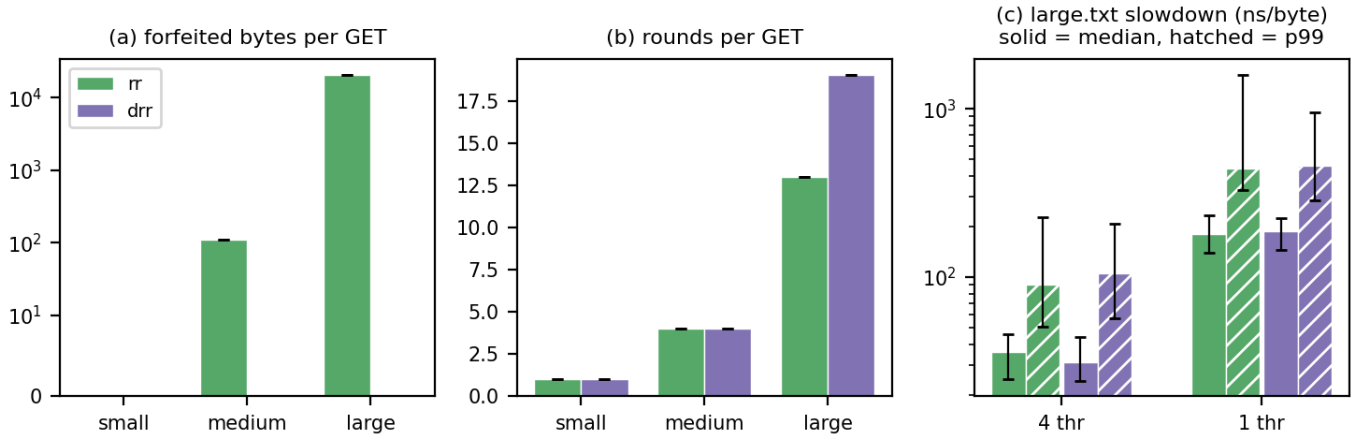
Reading the numbers. Median waiting falls from 649 us under fcfs to 481 us under sjf and to 190 / 187 us under rr / drr (about 3.4x lower), and with one server thread from 12.3 ms to 1.8 ms under rr: the preemptive policies keep the queue short for everyone because no request holds a worker for its whole transfer. Waiting is what the policy controls; the slowdown table shows who pays for it. The small class gains most: median slowdown 1037 (fcfs) to 644 (sjf) to 502 / 531 ns/byte (rr / drr), and with one server thread 11067 to 1924. The large class pays: 15 (fcfs), 23 (sjf), 36 / 31 (rr / drr); with one thread 101 under fcfs against 181 under rr. Medium files barely move at the reference configuration and gain under one thread. Throughput is lower under rr/drr (median 1923 / 2022 req/s vs 2471 for fcfs at the

reference configuration): every extra round costs a queue trip and extra system calls. That gap is comparable to the run-to-run spread (2471 [582-2835] for fcfs), so I treat it as a small cost rather than a firm ranking; with one server thread the throughputs (545 vs 631) are not separable.

4. rr versus drr (A29)

Table 3. Long-line comparison, median over 5 runs. A14 fired 1056 times per rr run at the reference configuration and 978 at one thread (range 882-1122 over all runs): exactly the 6 long lines of every large-file GET (about 176 GETs per run) and never under drr.

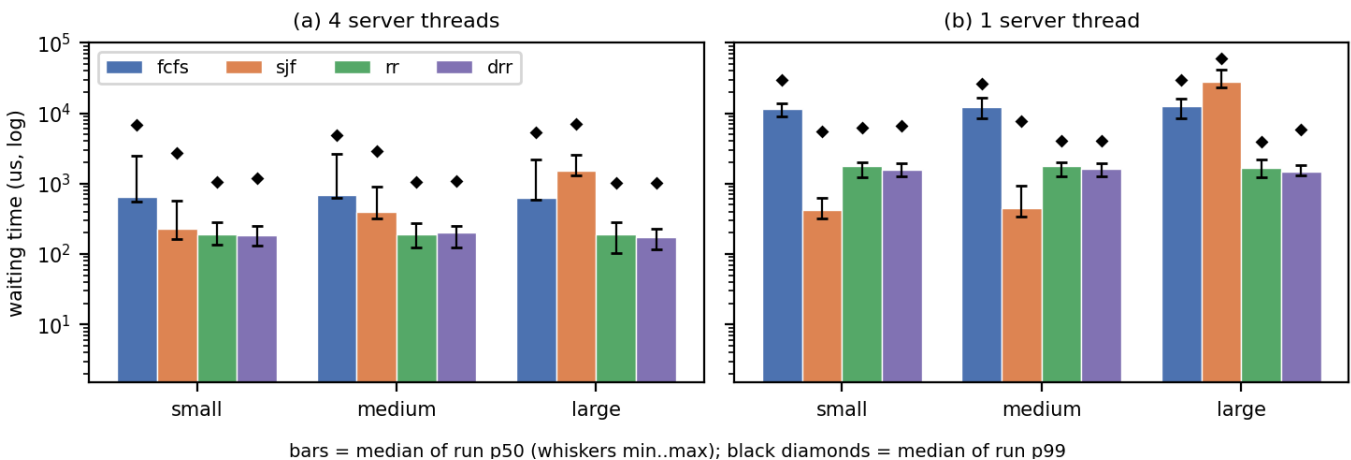
cell	forfeited bytes (all requests of a run)			rounds / GET		A14 fires	large.txt slowdown
	small	medium	large	medium	large	per run	median / p99
rr, 4 thr	0	18,040	3,570,864	4.0	13.0	1056	36 / 90
drr, 4 thr	0	0	0	4.0	19.0	0	31 / 105
rr, 1 thr	0	16,830	3,307,107	4.0	13.0	978	181 / 441
drr, 1 thr (suppl.)	0	0	0	4.0	19.0	0	187 / 460



Why long lines get less than their full allowance under rr. A round can only end on a line boundary. With ordinary 60-79 byte lines the leftover is tiny: a medium GET forfeits about 110 bytes in total over its 3 preempted rounds (under 0.5% of Q per round). A 20,000-byte line changes that. About 4.8 KB of ordinary lines separate the long lines, so when the next long line is reached mid-round it cannot fit in what is left, the round ends early and the remainder is forfeited; the round transfers only part of its Q. That happens once per long line: the large file forfeits 20,289 bytes per GET (3,570,864 in a run), which is about 3.4 KB, or 41% of a quantum, at each of its six long lines. The next round then starts with the long line and A14 sends it whole, overrunning Q by about 11.8 KB. rr's accounting is therefore uneven: it under-serves the long-line file before each long line and over-serves it on the A14 round, finishing the file in 13 rounds. **drr recovers exactly what rr forfeits:** the unused part of each allowance becomes deficit, so every round's allowance is Q plus what was left over and the file's cumulative service tracks Q per round. It never forfeits, never fires A14, and needs 19 rounds (= ceil(153,613 / 8192)), of which the deficit-building rounds ahead of each long line transfer nothing. What drr buys is exact, predictable accounting; what it costs is more rounds (and queue trips) for the long-line file.

Effect on response time. Here the two are not separable: the large file's median slowdown is 36 (rr) and 31 (drr) ns/byte at the reference configuration and 181 vs 187 with one thread, with the run-to-run ranges overlapping (fig. 3c). The reason is load: with 8 closed-loop clients there are never more than 4 requests waiting at the reference configuration (7 with one thread), so an extra round, or a zero-progress deficit round, is immediately followed by another turn. The mechanism differences above are large and deterministic; their effect on latency would need a longer queue than this workload produces.

5. Trade-offs and the SJF starvation



Who each policy penalises. *fcfs* is fair in waiting (all classes wait alike: 648 / 684 / 625 us at p50) but penalises *small* requests, which queue behind long transfers: their slowdown is the highest in every configuration (11067 ns/byte at one thread). *sjf* reverses this: with one thread it gives the best small and medium slowdown (641 and 131 ns/byte, against 1924 and 275 for *rr*) and pushes the cost onto the *large* class (196 ns/byte, about as much as *rr* / *drr* at 181 / 187 and about 1.9x *fcfs*). At the reference configuration *rr* / *drr* already match *sjf* for small files (502 / 531 vs 644). ***rr* and *drr*** get a similar small-file gain without ever ranking requests by size: they cut small-file slowdown by about 6x at one thread while charging the large class about 1.8x its *fcfs* slowdown, because a large request is now interleaved with everyone else instead of running to completion. What separates *sjf* is waiting time, next.

The SJF starvation (A11). Under *sjf* the wait depends on size: at the reference configuration the median wait is 229 us for small, 398 for medium and 1518 us for large requests (large waits 6.6x longer than small; under *fcfs*, *rr* and *drr* the three classes wait about the same). The tail shows it: with one thread the large class's p99 waiting time is 61 ms under *sjf* against 30 ms under *fcfs*, and *sjf* has the largest overall p99 waiting time in that configuration (57 ms vs 29 ms). Starvation here is bounded, because each of the 8 clients has at most one request outstanding, so a large request is overtaken only by requests issued while it waits; with an open arrival stream of small requests it would be unbounded, which is why real systems add aging.

6. Aside: --p (A30) and caveats

--p changes system calls, not scheduling. *fcfs* at the reference configuration with --p 1 vs --p 10 (3 alternating pairs): send() calls fell from about 158,466 to 18,108 per run (8.8x fewer) and median waiting fell from 860 to 491 us (lower in every pair). Throughput was too noisy to rank (medians 2147 vs 2252 req/s, single runs 677-3116). The scheduling columns are unaffected (rounds = 1, forfeited_bytes = 0 in both). **Caveats.** One machine, loopback, three workload files and a closed loop of 8 clients: the queue is short, which limits how far the policies can separate. Throughput and tail values move a lot between runs on this host, so the report relies on medians and on comparisons made inside the same block of runs. The A14 reading above is my interpretation of the spec; the alternative (apply it to a long line anywhere in a round) would leave *rr* with almost no forfeiture on this workload and remove the effect described in section 4.